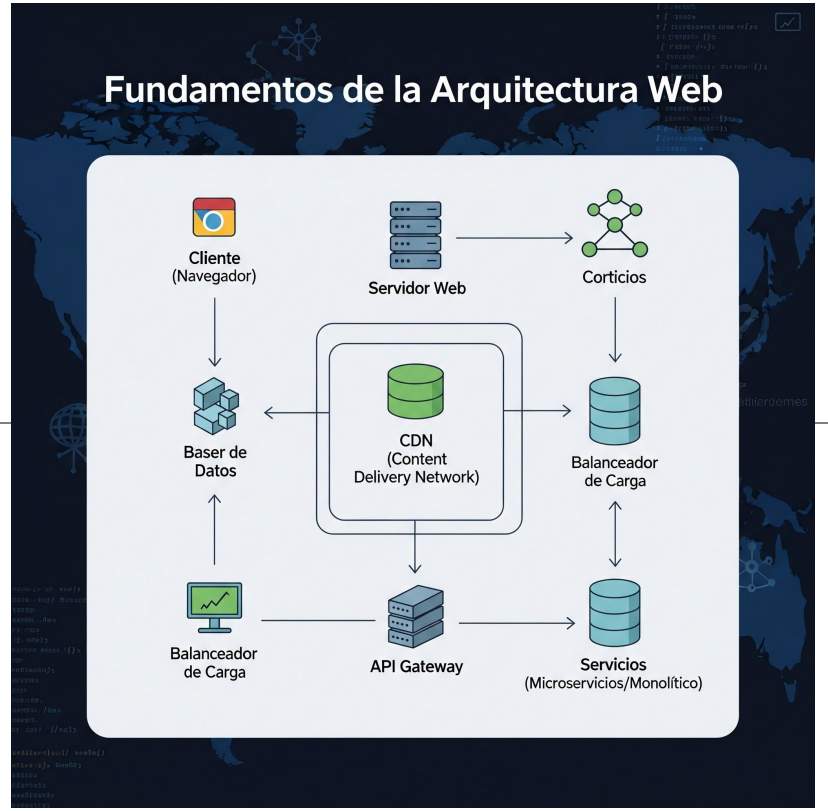
UD01. Fundamentos de la Arquitectura Web



Licencia

Toda la documentación de esta asignatura queda recogida bajo la licencia de Creative Commons



https://creativecommons.org/licenses/by-nc-nd/4.0/

En el caso de incumplimiento o infracción de una licencia Creative Commons, el autor, como con cualquier otra obra y licencia, habrá de recurrir a los tribunales. Cuando se trate de una infracción directa (por un usuario de la licencia Creative Commons), el autor le podrá demandar tanto por infracción de la propiedad intelectual como por incumplimiento contractual (ya que la licencia crea un vínculo directo entre autor y usuario/licenciatario). El derecho moral de integridad recogido por la legislación española queda protegido aunque no aparezca en las licencias Creative Commons. Estas licencias no sustituyen ni reducen los derechos que la ley confiere al autor; por tanto, el autor podría demandar a un usuario que, con cualquier licencia Creative Commons, hubiera modificado o mutilado su obra causando un perjuicio a su reputación o sus intereses. Por descontado, la decisión de cuándo ha habido mutilación y de cuándo la mutilación perjudica la reputación o los intereses del autor quedaría en manos de cada Juez o Tribunal.

El Viaje de una Petición Web

¿Qué ocurre realmente cuando escribes www.google.es y pulsas Enter?



El Viaje de una Petición Web

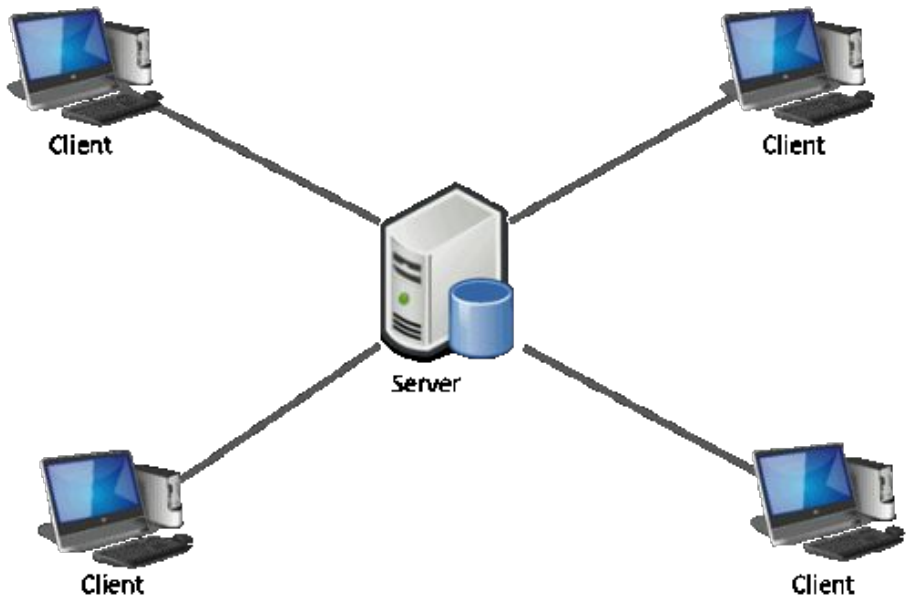
¿Qué ocurre realmente cuando escribes www.google.es y pulsas Enter?

El resumen:

1. Tu **navegador (Cliente)** pide la página.
2. La petición viaja por **Internet**.
3. Un **ordenador potente (Servidor)** la recibe y la procesa.
4. El servidor envía una **respuesta** que tu navegador dibuja en la pantalla.

El Protocolo HTTP/HTTPS

- Para que cliente y servidor se entiendan, necesitan un idioma común: **HTTP** (***HyperText Transfer Protocol***).
- Es un protocolo de **petición-respuesta**. El cliente pide, el servidor responde.
- **HTTPS** es la versión segura. La "S" significa que la conversación va **cifrada**.



Ejemplo: Cuando metes los datos de tu tarjeta en Amazon, **gracias a HTTPS**, viajan como un **mensaje secreto que solo el servidor de Amazon puede leer**.

Anatomía de una Petición HTTP

Una petición HTTP tiene 3 partes principales:

1. **Línea de Inicio (Start Line):** Contiene el **Método** (ej: GET), la **URL** del recurso (ej: /pagina.html) y la **versión** del protocolo (ej: HTTP/1.1).

Anatomía de una Petición HTTP

Una petición HTTP tiene 3 partes principales:

1. **Línea de Inicio (Start Line):** Contiene el **Método** (ej: GET), la **URL** del recurso (ej: /pagina.html) y la **versión** del protocolo (ej: HTTP/1.1).
2. **Cabeceras (Headers):** Metadatos sobre la petición. Información útil como el tipo de navegador (User-Agent), los formatos que acepta (Accept), etc.

Anatomía de una Petición HTTP

Una petición HTTP tiene 3 partes principales:

1. **Línea de Inicio (Start Line):** Contiene el **Método** (ej: GET), la **URL** del recurso (ej: /pagina.html) y la **versión** del protocolo (ej: HTTP/1.1).
2. **Cabeceras (Headers):** Metadatos sobre la petición. Información útil como el tipo de navegador (User-Agent), los formatos que acepta (Accept), etc.
3. **Cuerpo (Body):** Los datos que se envían al servidor. **¡Solo existe en algunas peticiones como POST!**

Métodos HTTP: Las Acciones

Las peticiones usan "**verbos**" para decirle al servidor qué acción quiere realizar.

- **GET:** Dame datos. Ejemplo: **Al visitar** una noticia en *marca.com*, tu navegador hace un **GET**.
- **POST:** Toma estos datos para crear algo nuevo. Ejemplo: **Al publicar** una foto en Instagram, tu móvil hace un **POST**.

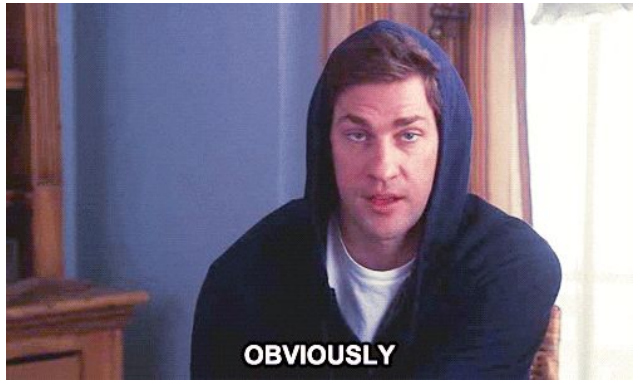
Métodos HTTP: Las Acciones

- **PUT / PATCH:** Toma estos datos para **actualizar algo que ya existe**. Ejemplo: Al cambiar tu biografía en tu perfil de Twitter/X, se usa un PUT o PATCH.
- **DELETE:** Borra este recurso. Ejemplo: Al **eliminar un correo** de tu bandeja de entrada de Gmail, se ejecuta un DELETE.

GET vs. POST: La Gran Diferencia

- **GET:**
  - Los datos viajan en la **URL** (ej: .../buscar?q=**libros**).
  - Es visible para el usuario, queda en el historial.

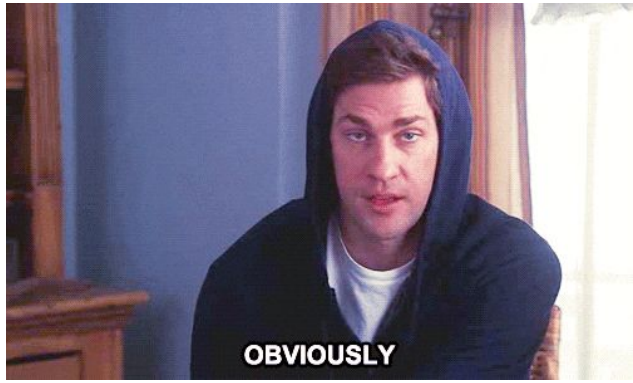
    ¡Nunca usar para contraseñas!
  - No tiene cuerpo (Body).
  - Es **idempotente**: puedes repetirlo mil veces y el resultado es el mismo.
- **POST:**
  - Los datos viajan **ocultos en el cuerpo** de la petición.
  - **Es más seguro** para enviar información sensible.
  - **No es idempotente**: si envías dos veces un formulario de registro, crearás dos usuarios.



GET vs. POST: La Gran Diferencia

- **GET:**
  - Los datos viajan en la **URL** (ej: .../buscar?q=**libros**).
  - Es visible para el usuario, queda en el historial.

    ¡Nunca usar para contraseñas!
  - No tiene cuerpo (Body).
  - Es **idempotente**: puedes repetirlo mil veces y el resultado es el mismo.



- **POST:**

En programación, una operación es idempotente si aplicarla varias veces produce el mismo resultado que aplicarla una sola vez.

  - **No es idempotente**: si envías dos veces un formulario de registro, crearás dos usuarios.

Códigos de Estado:
La Respuesta del Servidor

El servidor siempre responde con un **código numérico** para indicar cómo ha ido la petición. Se agrupan en familias:

- **1xx (Informativas):** Petición recibida, continuando proceso. (**Raras de ver**).
- **2xx (Éxito):** La petición fue recibida, entendida y aceptada con éxito. ✅
- **3xx (Redirección):** Se necesita tomar una acción adicional para completar la petición. ↪️
- **4xx (Error del Cliente):** La petición tiene una sintaxis incorrecta o no puede ser completada. ¡La culpa es tuya! 🤷‍♂️
- **5xx (Error del Servidor):** El servidor falló al intentar completar una petición aparentemente válida. ¡La culpa es mía! 🔥

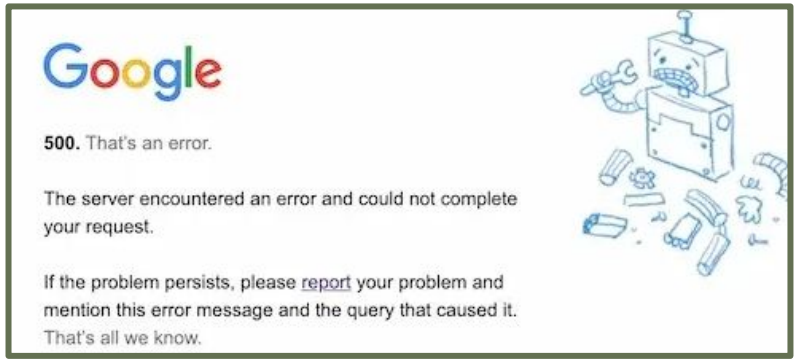
Códigos Comunes (Éxito y Redirección)

- **200 OK:** La respuesta estándar para peticiones correctas.
- **201 Created:** La petición ha tenido éxito y se ha creado un nuevo recurso como resultado. (Típica respuesta a un POST).
- **301 Moved Permanently:** La URL del recurso ha cambiado permanentemente. El navegador recordará la nueva dirección.
  - Ejemplo: Si la web del instituto **cambia de ies-antiguo.es a ies-nuevo.es**, se configura un 301 para que los usuarios y Google vayan al sitio nuevo automáticamente.

Códigos Comunes (Errores)

- **403 Forbidden:** Sé quién eres, pero no tienes permiso para ver esto.
- **404 Not Found:** No encuentro lo que me pides.
  - **Cuándo lo ves:** Al hacer clic en un enlace roto o al escribir mal una URL.





- **500 Internal Server Error:** Fallo genérico del servidor.
  - **Cuándo lo ves:** Cuando una web grande como Twitter o Facebook "se cae" y muestra un mensaje de error.

El Servidor Web

- El "**recepcionista**" de nuestro sistema.
- Su principal tarea es entregar **contenido estático** (HTML, CSS, imágenes).
- Es el primer punto de contacto para todas las peticiones entrantes.



Apache vs. Nginx: Los Titanes

- **Apache**: El **veterano fiable**, muy configurable (.htaccess).
- **Nginx**: El **aspirante moderno**, conocido por su increíble **rendimiento**.

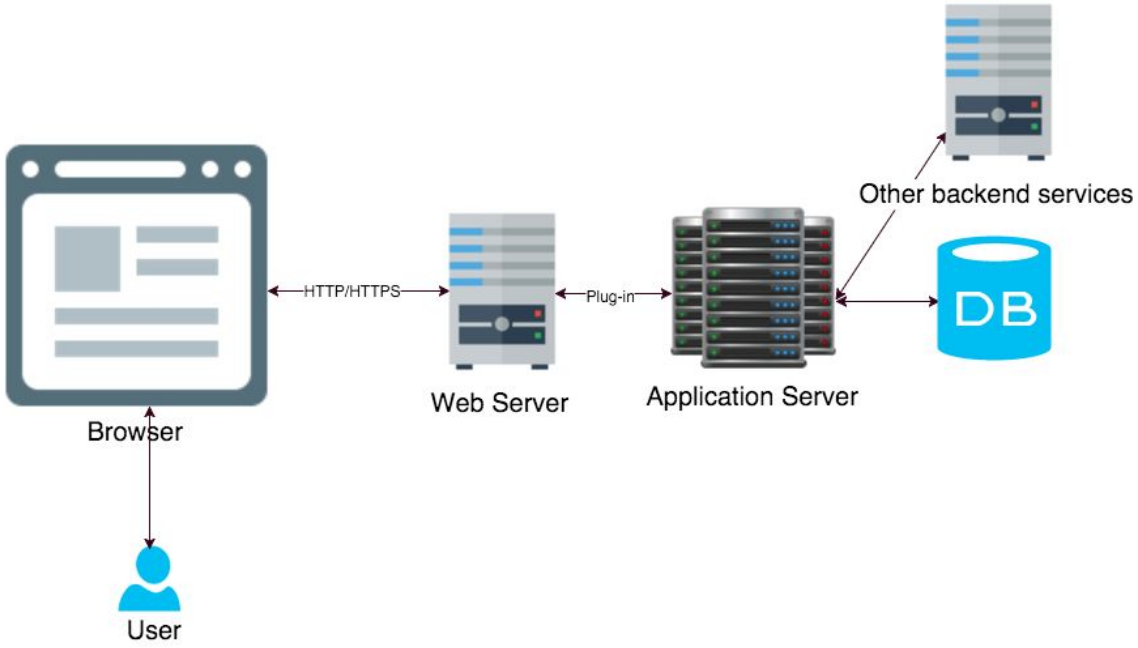
Ejemplo: En la web de Zara, cuando cargas la página de un producto, el servidor web (**Nginx**) te envía instantáneamente las fotos del producto, el logo de Zara y los ficheros CSS que le dan estilo a la página.

¡VEREMOS LOS DOS!



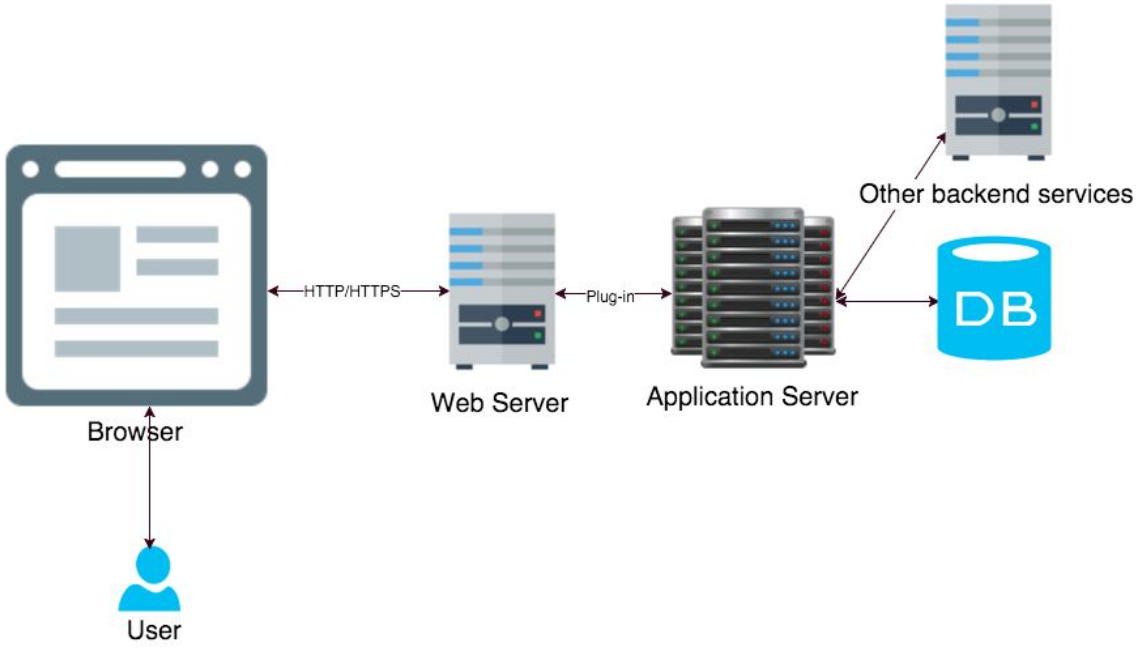
El Servidor de Aplicaciones

- El "**cerebro**" que ejecuta la **lógica de negocio** **cuando una petición es dinámica**.
- **Ejemplo:** En la misma página de Zara, cuando haces clic en "Añadir a la cesta", esa petición va al servidor de aplicaciones. Este ejecuta el código que verifica el stock y actualiza tu cesta.



El Servidor de Base de Datos

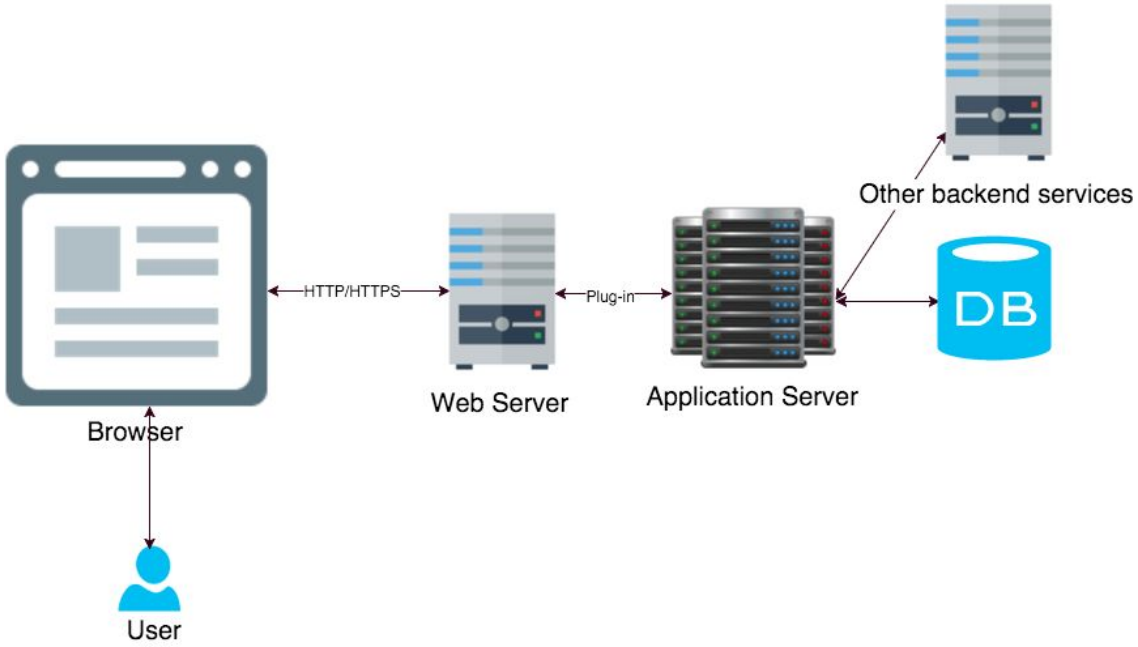
- La "**memoria**" que **almacena y organiza los datos de forma persistente**.
- **Ejemplo**: Cuando entras en la app de tu banco, el servidor de aplicaciones le pregunta al servidor de base de datos: "¿Cuál es el saldo y los últimos movimientos de la cuenta ES...XXX?".



El Equipo Completo: Visión General

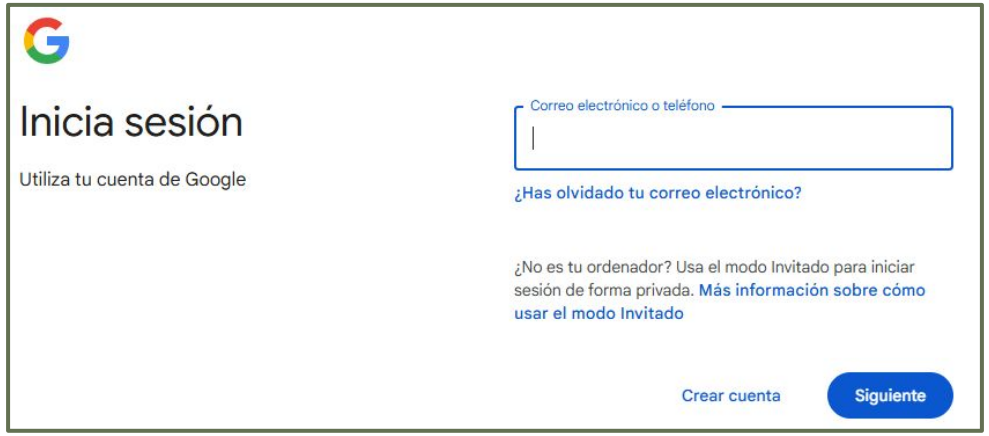
Ningún componente trabaja solo.

Forman un sistema interconectado donde cada pieza tiene una responsabilidad.



Flujo Detallado de un Login (Pasos 1-3)

1. **CLIENTE → Escribes gmail.com:** Tu navegador pregunta a un servidor **DNS** (la "agenda de teléfonos" de Internet) cuál es la dirección IP de gmail.com.
2. **DNS → Responde con una IP:** 142.250.200.133.
3. **CLIENTE → Petición a la IP:** Tu navegador envía la **petición HTTP** a esa dirección. Un **Firewall** en el borde de la red de Google es lo primero que la revisa.



Flujo Detallado de un Login (Pasos 4-6)

4. **FIREWALL** → **Petición permitida:** El firewall deja pasar la petición, que llega a un **Servidor Web (Nginx)**.

5. **SERVIDOR WEB** → **Es una petición dinámica:** El servidor web ve que es un POST a /login y se la pasa al **Servidor de Aplicaciones**.

6. **SERVIDOR DE APP** → **Consulta a la BBDD:** El servidor de app ejecuta el código de login y pregunta a la base de datos si el usuario y la contraseña son correctos.

Flujo Detallado de un Login (Pasos 7-9)

7. **BASE DE DATOS** → **Responde:** La base de datos valida los datos y responde: "Sí, el usuario es válido".

8. **SERVIDOR DE APP** → **Genera la respuesta:** Crea una sesión y genera el HTML de la bandeja de entrada.

9. **SERVIDOR WEB** → **Envía la respuesta:** Coge ese HTML y se lo envía al navegador del cliente con un código 200 OK.



¡Has iniciado sesión!

Arquitectura en 3 Capas

Es el modelo conceptual para organizar nuestro sistema:

- **Capa de Presentación (Frontend)**
- **Capa de Lógica/Negocio (Backend)**
- **Capa de Datos**



Profundizando en las 3 Capas

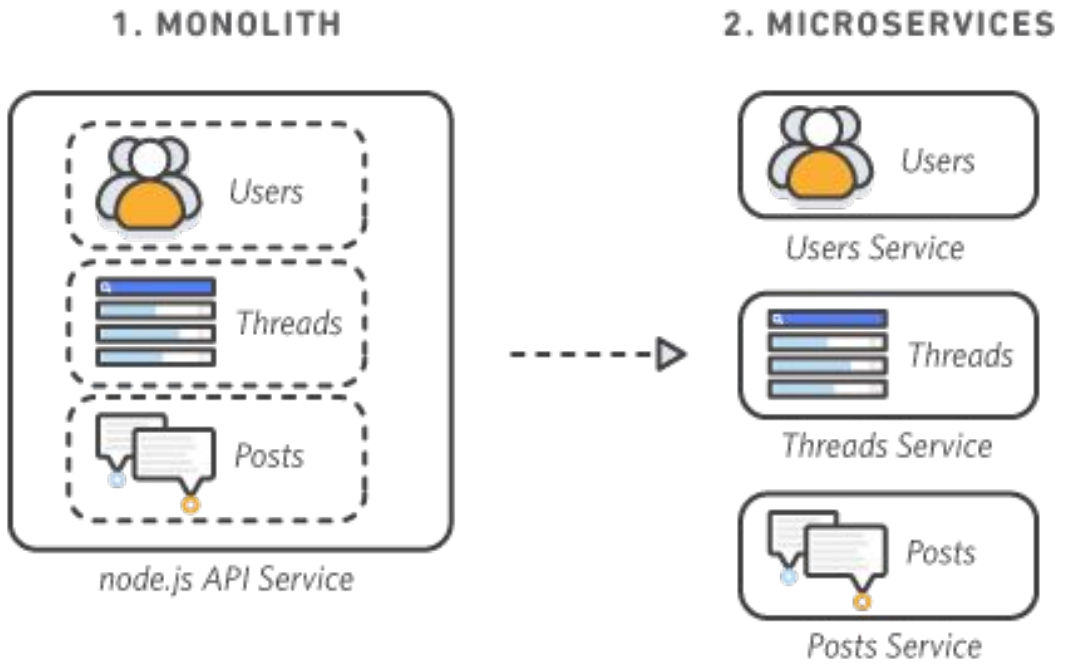
Ejemplo práctico con YouTube:

- **Capa de Presentación:** La página web o la app de YouTube **que ves** en tu móvil/PC.
- **Capa de Lógica:** El código en los servidores de Google que se encarga de **procesar** tu búsqueda, recomendarte vídeos o gestionar los "Me Gusta".
- **Capa de Datos:** Las gigantescas **bases de datos que almacenan** los vídeos, los comentarios y la información de todos los usuarios.



Monolitos vs. Microservicios

- **Arquitectura Monolítica:** Toda la aplicación en un **único bloque**.
- **Arquitectura de Microservicios:** La aplicación **se divide en pequeños servicios** independientes.



¿Cuándo usar Microservicios?

- Son ideales cuando la aplicación es **muy grande y compleja** y tienes **equipos de desarrollo grandes**.
- **Ejemplo: Netflix** es el rey de los **microservicios**.

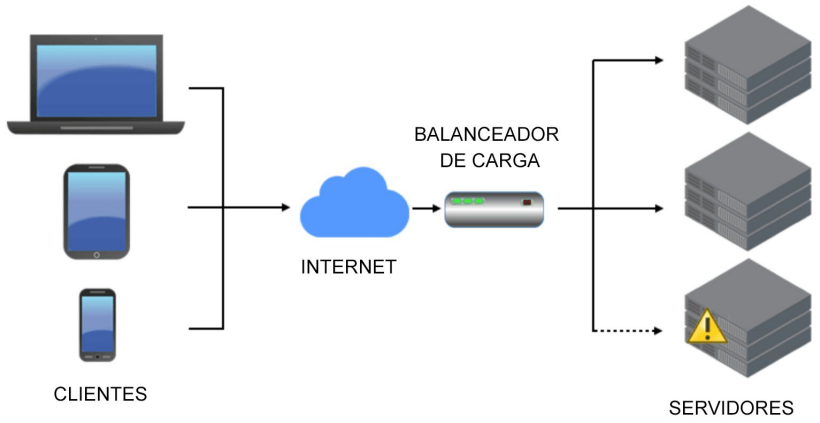
  Tienen un servicio para gestionar los perfiles de usuario, otro para el sistema de facturación, otro para las recomendaciones... Si el servicio de facturación falla, puedes seguir viendo series sin problema.



Balanceador de Carga

- Un "**repartidor de tráfico**" que **distribuye las peticiones entre varios servidores IDÉNTICOS** para que ninguno se sature.

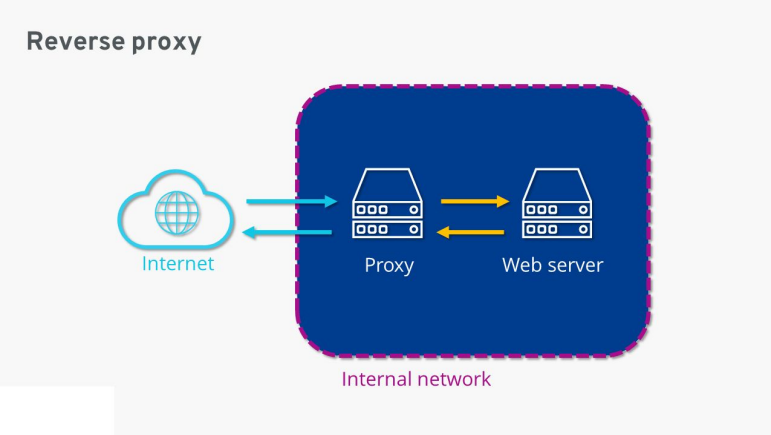
  Ejemplo: Durante el lanzamiento de las entradas para un concierto, la web de **Ticketmaster** recibe **miles y miles Y MILES de peticiones por segundo**. Un balanceador de carga distribuye estas peticiones entre decenas de servidores para que la web no se caiga.



Proxy Inverso

- Un servidor que **se sitúa delante de otros**, actuando como **intermediario**. Ofrece seguridad, cacheo y flexibilidad.

  Ejemplo: Un periódico como El Mundo puede usar un proxy inverso para que las peticiones a elmundo.es/deportes vayan a un servidor y las de elmundo.es/economia a otro distinto, aunque para el usuario sea la misma web.



HTTPS y Certificados SSL/TLS

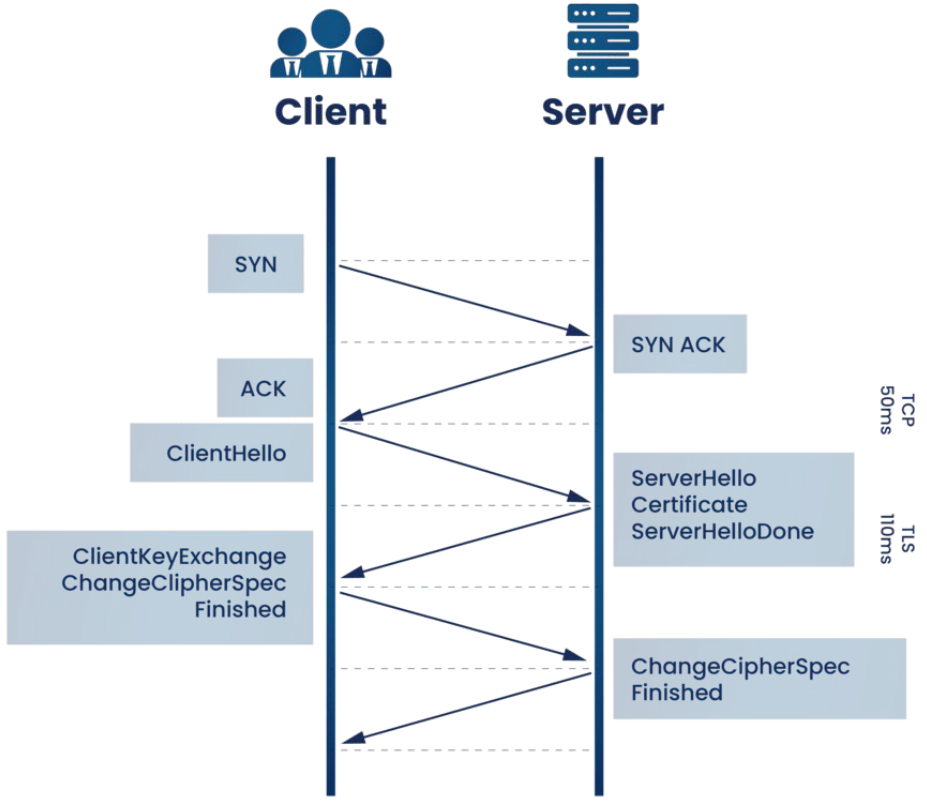
- La tecnología que **cifra la comunicación** entre cliente y servidor.

  Ejemplo: Sin HTTPS, si te conectas a una WiFi pública (aeropuerto, cafetería), **cualquiera en esa red podría ver tu contraseña** al iniciar sesión en una web. **Con HTTPS, solo verían datos cifrados.**



¿Cómo funciona el Handshake SSL/TLS? (Simplificado)

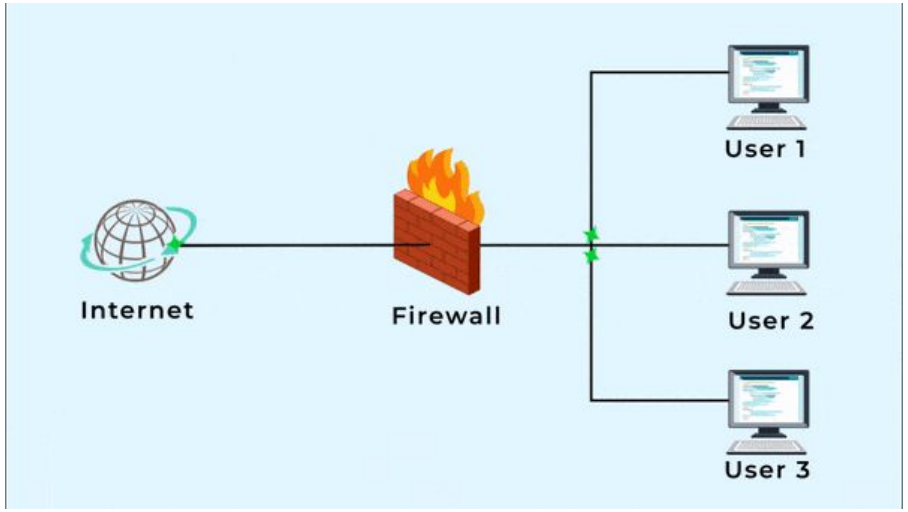
1. **Client Hello:** Tu navegador saluda al servidor.
2. **Server Hello:** El servidor responde y le envía su **Certificado Digital**.
3. **Verificación y Clave:** Tu navegador verifica el certificado y crea una clave de sesión única y secreta.
4. **Comunicación Cifrada:** Ambos usan esa clave secreta para cifrar toda la comunicación.



El Guardián: El Firewall

Una **barrera de seguridad** que controla el tráfico de red con **reglas**.

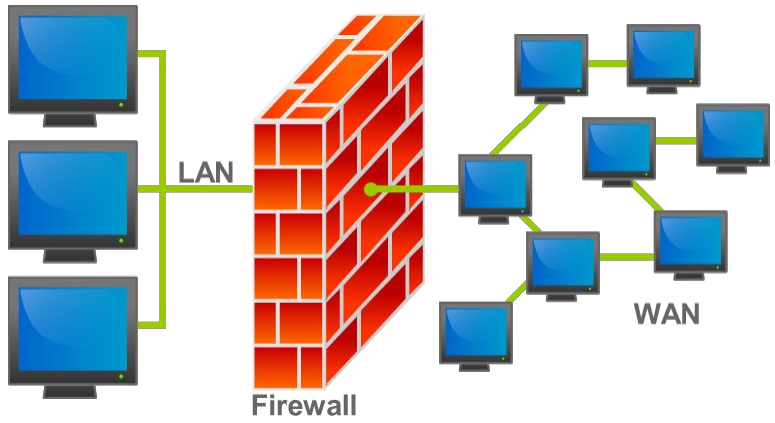
- **Ejemplo:** En la red de un instituto, el firewall tiene **reglas** como: "Permitir a los alumnos navegar por webs (puertos 80 y 443), pero bloquear el acceso a redes de videojuegos online (bloqueando sus puertos específicos)".



Tipos de Firewall

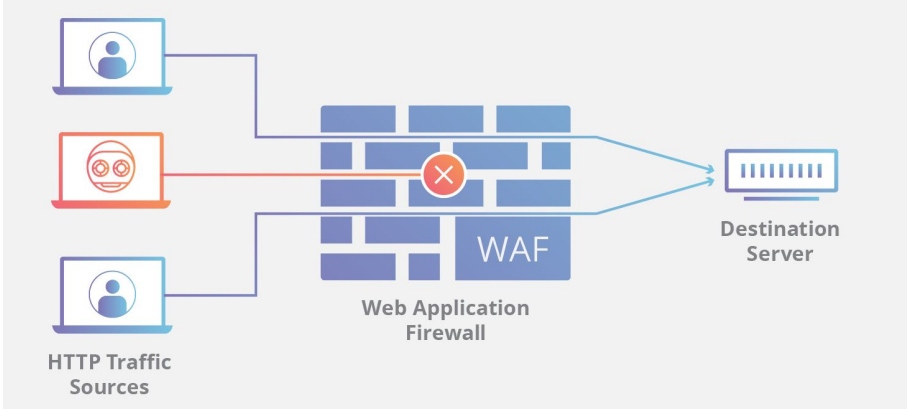
- **Firewall de Red:** Toma decisiones basadas en **IPs y puertos**.





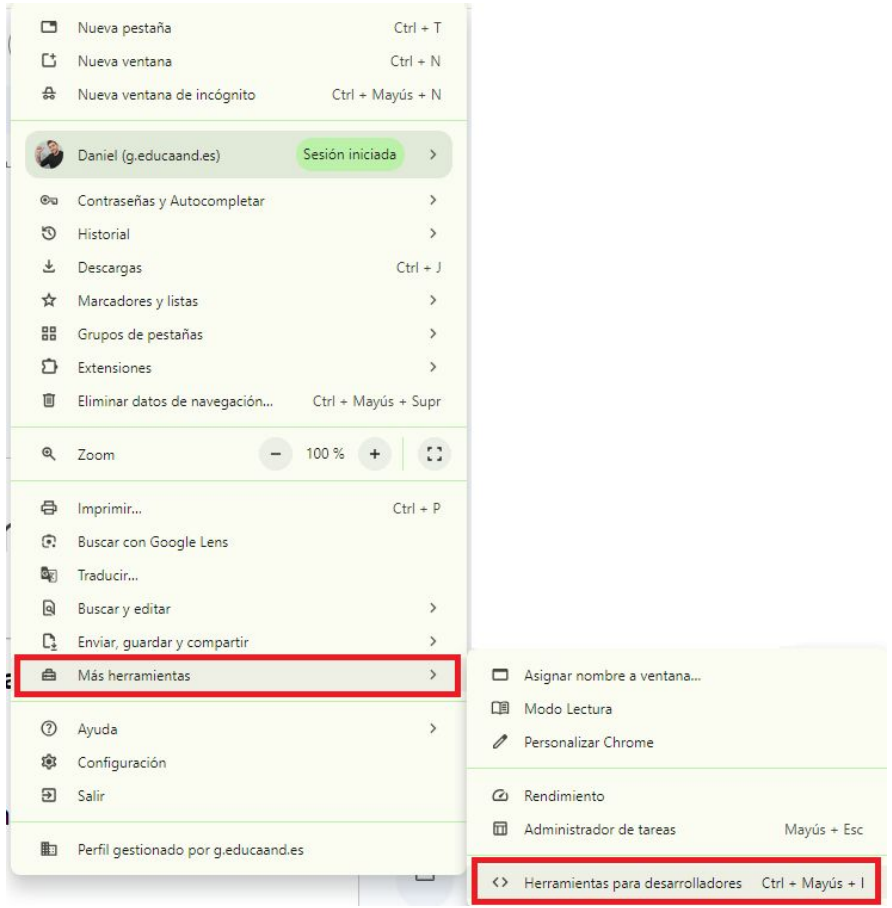
- **Web Application Firewall (WAF):** Un tipo más avanzado que **entiende HTTP** y puede bloquear ataques como **Inyección SQL**.

https://www.youtube.com/watch?v=DRsfRMAfY18&t=22s



Nuestra Caja de Herramientas: **DevTools**

- Todos los navegadores modernos incluyen un panel de **Herramientas de Desarrollador** (DevTools).
- Se abre pulsando Ctrl + Mayus + I o **F12**.
- Nos permite ver las "tripas" de una página: su código, su rendimiento y, lo más importante, **toda la actividad de red**.



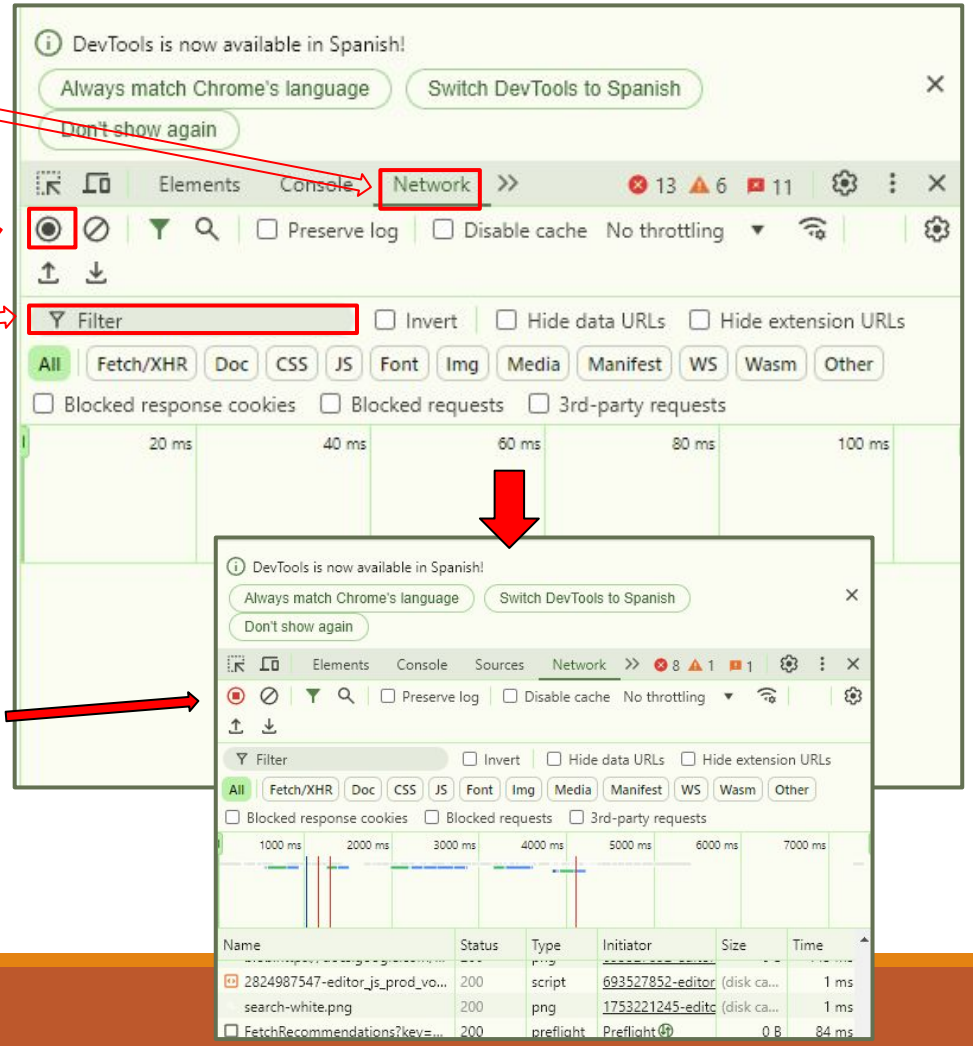
Paso a Paso: La Pestaña de Red

Haz clic en la pestaña "Red" o "Network".

Partes importantes:

1. **Botón de Grabar (🔴)**
2. **Filtro**
3. **Lista de Peticiones**
4. **Columnas: Nombre, Estado, Tipo…**

Una vez empiezas a grabar, si actualizas (F5)



Paso a Paso: Inspeccionando una Petición

Haz clic en cualquier petición de la lista para abrir el panel de detalles.

Pestañas clave en los detalles:

- **Headers (Encabezados):** Para ver la URL, el Método y el Código de estado.
- **Payload (Carga útil):** Para ver los datos que **enviaste** (en un POST).
- **Response (Respuesta):** Para ver lo que el servidor te **contestó**.



Resumen

Hemos tratado:

- El protocolo HTTP, sus métodos y códigos.
- Los roles y diferencias entre los servidores clave.
- El flujo de una petición y las arquitecturas más comunes.

En la siguiente unidad...

- ¡Dejaremos la teoría y pasaremos a la acción! **Instalaremos y configuraremos nuestro propio servidor web desde cero.** 